MoA Prediction

A Multi-Model Ensemble for Mechanism-of-Action Classification

Marco Merlonghi | Project Work in Machine Learning and Data Mining

About the Competition

- **Host:** Laboratory for Innovation Science at Harvard (LISH), with the Broad Institute's Connectivity Map (LINCS);
- **Platform:** Kaggle – `https://kaggle.com/c/lish-moa`;
- **Timeline:** launched and closed in 2020 (final deadline Nov. 30, 2020);
- **Scale:** several thousand teams (see the Kaggle leaderboard page);
- **Data origin:** L1000 assay – a high-throughput assay measuring how human cell lines respond transcriptionally to a chemical or genetic perturbation.

Why it matters

Mapping a compound's gene-expression and cell-viability signature to its mechanism of action accelerates **drug discovery** and **repurposing**, replacing slow, costly wet-lab characterisation with a computational first pass.

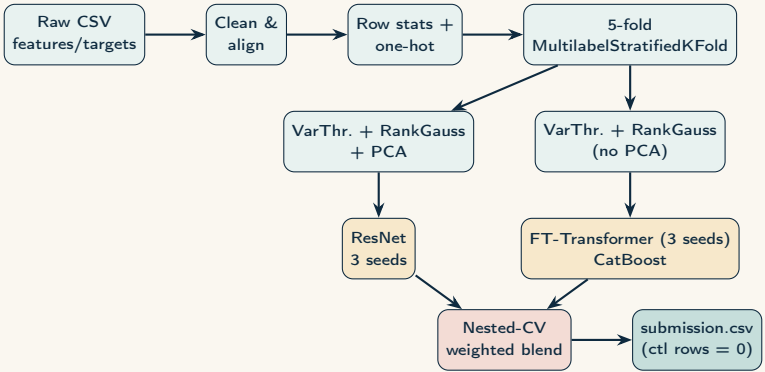
The Problem and the Data

- **Task:** multi-label classification of **206 mechanisms of action (MoA)**
- **Input:** 772 gene-expression (`g-`) and 100 cell-viability (`c-`) features, plus `cp_time` and `cp_dose`
- **Size:** ≈23.8k training rows, ≈22k after removing controls
- **Imbalance:** positives are very rare (mean predicted probability on treated test rows: 0.0037)
- **Metric:** mean multi-label binary log loss, clipped to avoid log(0)
- Control rows (`cp_type == ctl_vehicle`) carry **no MoA**: excluded from training, set to 0 at submission

Why an ensemble?

Dense nets and tree models fail differently on rare positives ⇒ blending several architectures on different preprocessing variants reduces variance.

Pipeline at a Glance



Setup, Data Cleaning and Hyperparameters

- `iterative-stratification` for multi-label stratified K-fold
- `PYTORCH_CUDA_ALLOC_CONF` to reduce GPU memory fragmentation
- **5 folds**, **3 seeds** $\{42, 202, 2024\} \Rightarrow$ bagging
- AMP enabled only if a GPU is present

```
N_FOLDS = 5
SEEDS = [42, 202, 2024]
EPOCHS = 35 # AdamW, lr 1e-3
WEIGHT_DECAY = 1e-5
EARLY_STOPPING_PATIENCE = 8
INNER_VAL_FRACTION = 0.1
LABEL_SMOOTHING = 0.001
PCA_G_COMPONENTS = 50
PCA_C_COMPONENTS = 15
CATBOOST: 250 iters, depth 5,
       lr 0.08
```

- `load_raw_data`: reads `train_features`, `train_targets_scored`, `test_features`, `sample_submission`
- `clean_and_align`: drops NaNs/duplicates, keeps the `sig_ids` present in both tables, sorts by `sig_id`
- **Strict alignment check** before proceeding:

```
assert features.shape[0] == targets.shape[0]
assert (features["sig_id"].values
      == targets["sig_id"].values).all()
```

Feature Engineering

Row statistics (separately for `g-` and `c-` blocks):

- sum, mean, std, kurtosis, skewness
- captures the overall activation level per sample

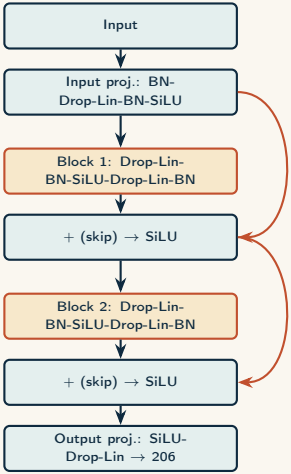
```
df["g_sum"] = df[g_cols].sum(axis=1)
df["g_mean"] = df[g_cols].mean(axis=1)
df["g_std"] = df[g_cols].std(axis=1)
df["g_kurtosis"] = df[g_cols].kurtosis(axis=1)
df["g_skew"] = df[g_cols].skew(axis=1)
# ... mirrored for c_cols
```

Categorical encoding:

- one-hot on `cp_time` and `cp_dose`
- test columns realigned to the training reference columns

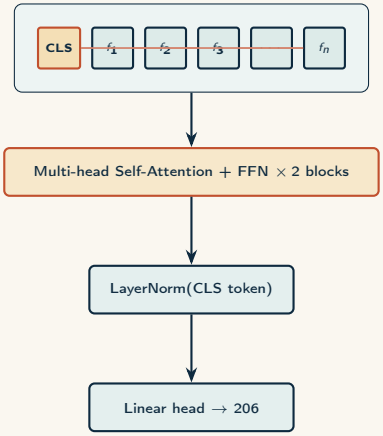
```
def encode_categoricals(df, dummy_cols,
                        reference_columns=None):
    df = pd.get_dummies(df, columns=dummy_cols,
                        dtype=np.float64)
    if reference_columns is not None:
        df = df.reindex(columns=reference_columns,
                        fill_value=0)
    return df
```

TabularResNet – How It Works



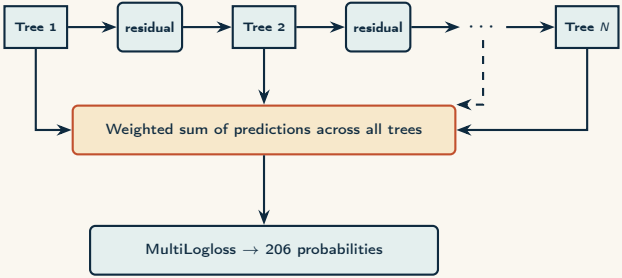
- The curved orange arrows are the **residual (skip) connections**: the block's input is added to its output
- Eases gradient flow, stabilises deeper networks
- 2 residual blocks, `hidden_dim = 512`, dropout 0.2
- Consumes **RankGauss + PCA** features: the transformed features *plus* 50 + 15 PCA components (≈950 inputs)

FT-Transformer – How It Works



- Each scalar feature → its own **token** (≈890 tokens/row incl. CLS)
- A learnable **[CLS]** token is prepended
- **Self-attention** (thin lines) connects every token to every other
- The CLS output summarises the row → classification head
- $O(L^2)$ cost ⇒ smaller batch size (64)
- Consumes **raw RankGauss** features (no PCA)
- Simplified variant of Gorishniy et al. (post-LayerNorm, GELU, no ReGLU)

CatBoost – How It Works



- **Gradient boosting**: each tree corrects the residual error of the previous ones
- A single multi-output `MultiLogloss` fit, not 206 separate fits
- Splits stay aligned to the original features → trained on **raw RankGauss** (no PCA)
- Fixed 250 iterations (no early stopping), one fit per fold, no seed bagging
- Targets constant within a fold are excluded from the fit and filled with the observed constant

Metric, Loss Variants and Training Routine

Evaluation metric

$$\mathcal{L} = -\frac{1}{N \cdot 206} \sum_{i=1}^{N} \sum_{j=1}^{206} \left[y_{ij} \log \hat{y}_{ij} + (1 - y_{ij}) \log(1 - \hat{y}_{ij}) \right]$$

predictions clipped to $[\epsilon, 1-\epsilon]$, with $\epsilon = 10^{-15}$, computed in float64.

Optional variants (training loss only)

- **Focal loss:** down-weights easy examples (off by default)
- **Label smoothing** (0.001): $y(1-\epsilon) + 0.5\epsilon$
- Model selection **always** uses the plain, non-smoothed log loss

- **AMP** (`torch.amp.GradScaler`) for speed and stability
- **Early stopping** (patience = 8 epochs) and **ReduceLROnPlateau**, both on the inner split
- Seeds set *before* the model is built: initial weights depend on the seed
- **In-RAM checkpointing**: best `state_dict` cloned to CPU, written to disk once at the end
- Explicit `torch.cuda.empty_cache()` so sibling workers reclaim VRAM

Why it matters

Avoids dozens of slow disk writes per fold and prevents GPU memory creep across 30 jobs.

Validation Protocol

Outer loop: 5-fold OOF

`MultilabelStratifiedKFold`, seed 42. Each fold's validation part yields the out-of-fold predictions and is scored **once**, after training.

Inner split: model selection

10% of each training fold (stratified) drives early stopping, LR scheduling and best-epoch choice of the neural networks. CatBoost uses fixed iterations.

Blend: nested CV

Weights are fitted on $k-1$ folds' OOF predictions and scored on the held-out fold.

Known limitation

The data has no drug identifier, so the CV cannot be grouped by compound: the same compound may appear in training and validation. Scores can be optimistic compared with the leaderboard.

Two Preprocessing Pipelines

PCA variant (ResNet)

VarianceThreshold → RankGauss (`QuantileTransformer`) → PCA(50) on `g-`, PCA(15) on `c-` → **concatenate** transformed features, PCA components and raw dummies (≈950 columns)

Raw variant (FT-Transformer, CatBoost)

VarianceThreshold → RankGauss, **no PCA** – keeps one token/split per original feature (≈890 columns)

Both fitted **per fold** on the training split only and cached to disk as `.joblib`. The raw pipeline fitted for the transformer is reused by CatBoost within the same run; stale caches are cleared at the start of each run.

NN Cross-Validation at Scale

- 2 networks $\times$ 3 seeds $\times$ 5 folds $=$ **30 jobs**
- `joblib` + `loky` backend, GPUs assigned **round-robin**
- 2 GPUs $\times$ 2 jobs/GPU $=$ **4 parallel workers**
- Predictions averaged over the 3 seeds per (fold, model)
- Wall-clock: NN CV 96.3 min, CatBoost CV 29.7 min, inference 0.5 min

```
NN_JOBS_PER_GPU = 2
N_JOBS_NN = N_GPUS * NN_JOBS_PER_GPU

Parallel(n_jobs=N_JOBS_NN,
       backend="loky")(
 delayed(_train_nn_job)(...)
 for job in jobs
)
```

Ensemble Blending: Methodology

Weight search

- Softmax-reparameterised weights on the simplex
- **Nelder–Mead**, not SLSQP: avoids a zero finite-difference gradient stalling the optimiser at x_0

Nested CV

- Weights fit on $k-1$ folds, scored on the held-out fold
- Prevents an over-optimistic blend score
- Final weights = mean across folds

Two scores reported

naive OOF blend score (same data used to fit the weights) vs. *honest* nested-CV score

Results – Fold-by-Fold Validation Loss

Fold	ResNet	FT-Transformer	CatBoost
0	0.01686	0.01693	0.01707
1	0.01705	0.01710	0.01709*
2	0.01652	0.01656	0.01676
3	0.01679	0.01688	0.01704*
4	0.01700	0.01705	0.01704
Mean	0.01684	0.01690	0.01700

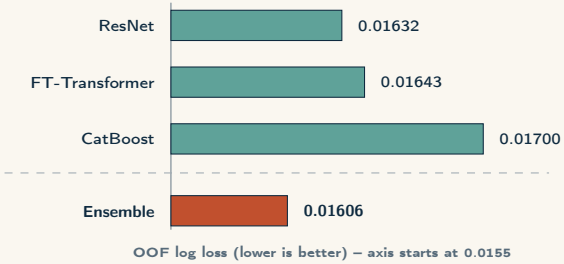
* 1 constant target in the training fold, excluded from the CatBoost fit and filled with the observed constant.

Network values are the *mean of the per-seed losses*; CatBoost is trained once per fold. The OOF scores on the next slide use *seed-averaged predictions*, which lowers the loss by about 0.0005 (bagging gain). ResNet is the best network in every fold.

Results – Out-of-Fold Summary

Model	OOF log loss
ResNet	**0.01632**
FT-Transformer	0.01643
CatBoost	0.01700
Ensemble (nested)	**0.01606**

Timing: NN CV 96.3 min | CatBoost CV 29.7 min | total 128.6 min



Results – Ensemble Blend (Nested CV)

Score and weights per held-out fold

Fold	Score	w_{resnet}	w_{ft}	w_{cb}
0	0.01603	0.447	0.374	0.179
1	0.01618	0.460	0.369	0.170
2	0.01570	0.437	0.373	0.190
3	0.01617	0.428	0.389	0.183
4	0.01622	0.465	0.370	0.166

Final weights (mean across folds)

ResNet 44.7% | FT-Transformer 37.5% | CatBoost 17.8%

Honest nested-blend score

0.01606 ± 0.00019

Naive OOF blend (0.01606) equals the nested score. With only three weights this is expected; it does not rule out leakage from the CV design (no grouping by compound).

Ablation – Is an Extra Network Worth It?

Base: ResNet + FT + CatBoost	Extra OOF	Ensemble (nested)	Extra weight
+ MLP (PCA features)	0.01649	0.01604	1.5%
+ 1D-CNN (PCA features)	0.01661	0.01605	8.2%
+ 1D-CNN (raw features)	0.01661	0.01606	7.0%
no extra network (final)	–	**0.01606**	–

Reading

All four ensembles score 0.01604–0.01606: the differences ($\leq 2 \cdot 10^{-5}$) are below the fold standard deviation (0.00018) and the run-to-run GPU noise ($\approx 10^{-5}$).

Decision

The extra network earns weight but no measurable gain, even with raw features. The simpler 3-model blend is kept: fewer components to train, explain and maintain.

Conclusions, Limitations and Future Work

Findings

- Ensemble 0.01606 vs. best single model (ResNet) 0.01632: ≈1.6% better
- ResNet and FT-Transformer carry ≈82% of the weight; CatBoost, weakest alone, gets 18%
- An extra network (MLP or 1D-CNN, on PCA or raw features) does not improve the blend

Future work

- Group the CV by compound; measure the leaderboard score
- Ablations: PCA vs. raw for the ResNet, error correlation between models

Limitations

- No compound identifier: possible leakage across folds, optimistic scores
- The inner split removes 10% of the training data; an earlier protocol that selected epochs on the OOF fold gave 0.01595, so about 0.0001 of that gap was optimism
- GPU non-determinism: repeated runs differ by $\approx 10^{-5}$; wall-clock time varied by about 20% across runs
- Why CatBoost earns weight (diverse errors) was not measured

References

- Kaggle, *Mechanisms of Action (MoA) Prediction*, 2020. https://www.kaggle.com/competitions/lish-moa
- Gorishniy et al., *Revisiting Deep Learning Models for Tabular Data*, NeurIPS 2021 (FT-Transformer).
- Prokhorenkova et al., *CatBoost: unbiased boosting with categorical features*, NeurIPS 2018.
- Sechidis, Tsoumakas, Vlahavas, *On the Stratification of Multi-Label Data*, ECML PKDD 2011.
- Bradberry, *iterative-stratification* (Python package). https://github.com/trent-b/iterative-stratification
- Paszke et al., *PyTorch*, NeurIPS 2019; Pedregosa et al., *Scikit-learn*, JMLR 12, 2011.
- Loshchilov, Hutter, *Decoupled Weight Decay Regularization* (AdamW), ICLR 2019.
- Micikevicius et al., *Mixed Precision Training*, ICLR 2018.
- Lin et al., *Focal Loss for Dense Object Detection*, ICCV 2017; Szegedy et al., *Rethinking the Inception Architecture* (label smoothing), CVPR 2016.
- Nelder, Mead, *A simplex method for function minimization*, The Computer Journal 7(4), 1965.